

Concurrency Control

Type : 1. Multiversion Timestamp Ordering (MVTO)

2. Multiversion Two-Phase Locking Using Certify Lock (MV2PL) (C-Lock)

3. Two Phase Lock (2PL)

4. Graph Based Concurrency control

Multiversion
Concurrency
control

MV2PL

T1 T2 $x \rightarrow x_1, x_2, x_3$
↘ ↙

2PL

Share Lock (Read) X

Exclusive Lock (Read/Write) ↙

3. Two Phase Lock (2PL)

* Transactions must get a lock before read or write.

* Only transactions compatible can run at the same time.

1. Share Lock (S-Lock): Read only

2. Exclusive Lock (X-Lock): Read + Write

	S	X
S	✓	✗
X	✗	✗

Phase :

1. Growth Phase :

* Lock takes place

* Upgrading of lock takes place ($S \rightarrow X$)
read modify

* No lock release

2. Lock Point : After getting a final lock

3. Shrinking Point : * No locking takes place

* Only unlocking

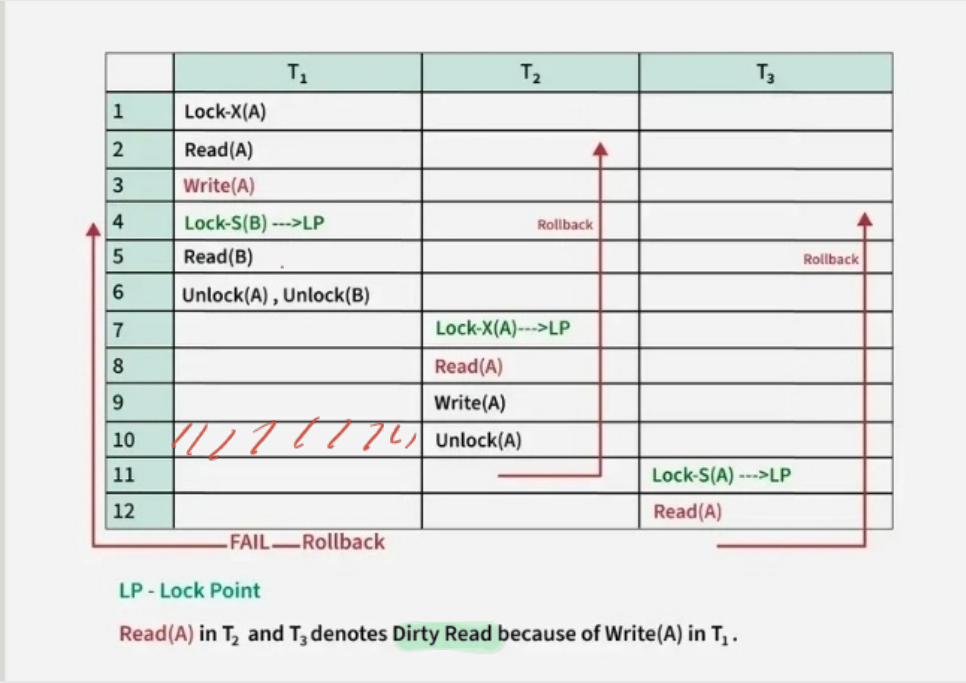
* Downgrading a lock ($X \rightarrow S$)
modify read

Question: *A → S lock → X lock*
B → X lock

- 1 Lock-S(A) -> Read(A) -> Lock-X(B) -> Upgrade(A) -> Write(A) -> Unlock(B) -> Downgrade(A) -> Read(A) -> Unlock(A)
- a. Identify the exact operation that serves as the Lock Point for T1.
 - b. Identify the Growing Phase and Shrinking Phase ranges.
 - c. Explain why attempting to execute Lock-S(C) right after Unlock(B) would violate basic 2PL rules.

Ans:

- (a) Lock Point: Upgrade(A) (or Write(A)), as this is the final lock acquisition or escalation operation before the first unlock occurs.
- (b) Growing Phase: From Lock-S(A) up to Upgrade(A).
Shrinking Phase: From Unlock(B) to Unlock(A).
- (c) Under 2PL, once a transaction enters the shrinking phase by releasing its first lock (Unlock(B)), it can never acquire any new lock. Acquiring Lock-S(C) after unlocking an item violates the fundamental 2PL rule.



cascading rollback undo

S-lock

X-lock

Categories of 2PL:

(1) Conservative 2PL: * Things the transaction needs are locked (acquire a lock), before execution [Read/Update]

* Deadlock-free

* Serializable

* Less Practical

(2) Strict 2PL: * Basic

- * X-lock is kept till committed
- * Prevent dirty read
- * No cascading roll back

(3) Rigorous 2PL: Both X-lock and S-lock are acquired

Match each scenario with its corresponding category

Scenario A:

Transaction T1 releases its shared read locks on table X immediately after reading, but holds all exclusive write locks on table Y until it commits to prevent dirty reads.

Scenario B:

Transaction T2 requests and acquires locks for data items A, B, and C before executing any operations. If item C is locked by another transaction, T2 acquires no locks and waits.

Scenario C:

Transaction T3 holds all of its shared read locks and exclusive write locks until it finishes and commits.

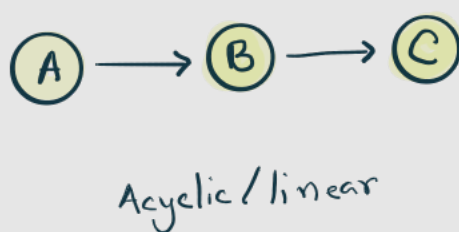
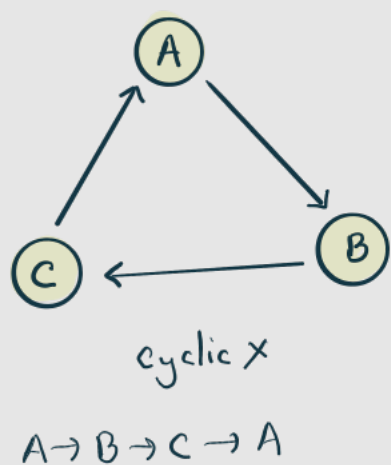
] strict 2PL

] Conservative 2PL

] Rigorous 2PL

————— X —————

4. Graph Based Concurrency Control



Advantage: 1. Conflict serializable
2. Deadlock-free

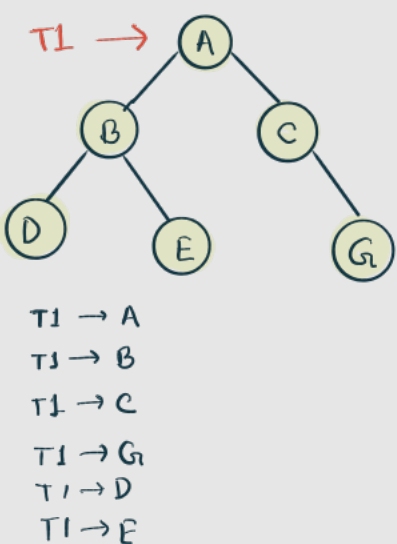
Limitation: 1. Cascading roll back
2. Order must be predefined.

Tree Based Concurrency Control

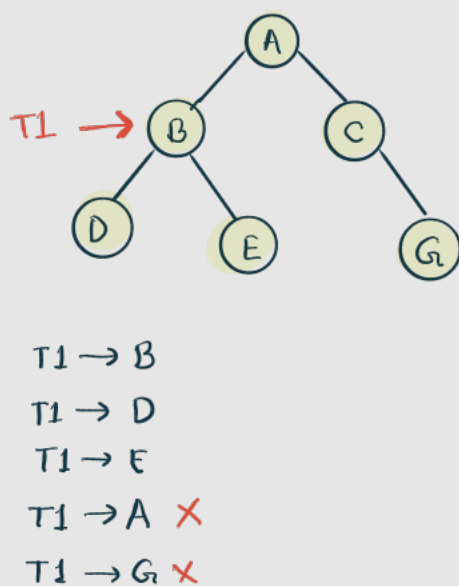
↳ only X-lock

Rules: 1. For the first time, one transaction can lock any node.
2. Then it can only lock its children

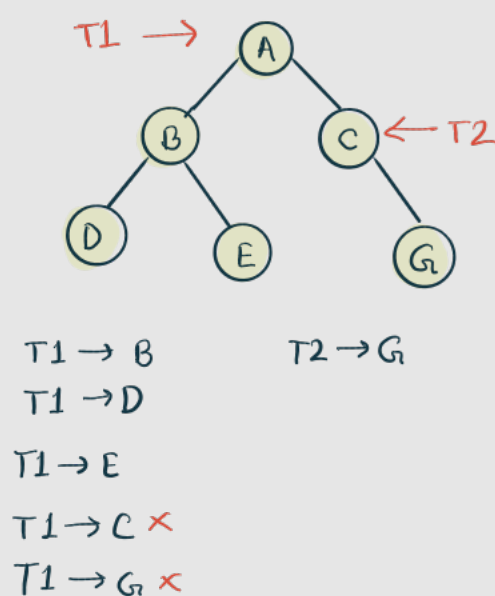
Ex 1



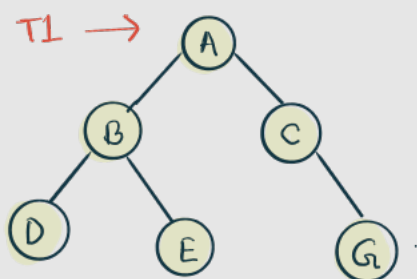
Ex 2



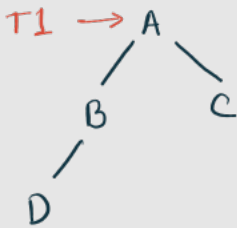
Ex 3



Ex 4



Given a database organized in a tree hierarchy where Data Item A is the root, B and C are children of A, and D is a child of B:



a. Transaction T1 needs to modify item D. List the minimal sequence of lock operations T1 must perform under the Tree Protocol.

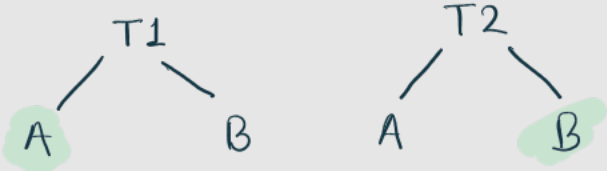
b. Can T1 perform a Read operation using a Shared Lock (S-lock) in this protocol? Explain why or why not.

No

c. State whether the Tree Protocol guarantees deadlock-free execution and if it prevents cascading rollbacks.

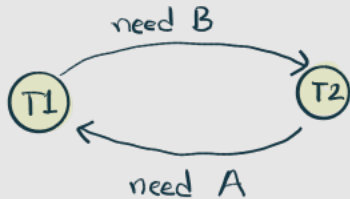
DeadLock

→ Performance reduced
→ makes the pc slow



Condition:

1. Hold and wait
2. No preemption/force
3. Circular wait



Ways:

1. Avoid / Prevent (Before)
2. Detect (When) — *passive strategy*
3. Recovery (After)

① Avoid:

(a) Follow same order:

5 T1 → A, B
10 T2 → A, B

①
T1 → A
T2 → A ✗

②
T1 → B
T2 → B ✗

(b) Smaller Lock/Isolation level:

* Isolation

T1 → A(grade), B
T2 → A(name), B

name	id	grad		

A

(c) Wait-Die Scheme: Non-preemptive * Remember

[Timestamp based]

- T1 can wait if it wants T2's resource
- T2 is aborted and restarted if requests for resource held by T1.

Example:

- T1 (older) holds Account A
- T2 (younger) holds Account B

Case 1:

- T1 wants Account B → T1 waits

Case 2:

- T2 wants Account A → T2 is stopped and restarted

This avoids deadlock because younger transactions are not allowed to wait and form a cycle.

T1 (old) — Wait
T2 (younger) — Die

(d) Wound-Wait Scheme: preemptive * Remember

[Timestamp based]

- T1 can force T2 to stop
- T2 must wait if they want a resource held by T1.

Example:

- T1 (older) holds Account A —
- T2 (younger) holds Account B —

Case 1:

- T1 wants Account B → T2 is stopped, T1 continues

Case 2:

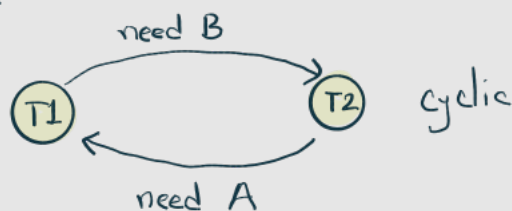
- T2 wants Account A → T2 waits

This avoids deadlock because older transactions are never blocked by younger ones.

T1(5) — wound
T2(10) — wait

② Detect:

(a) Wait for Graph:

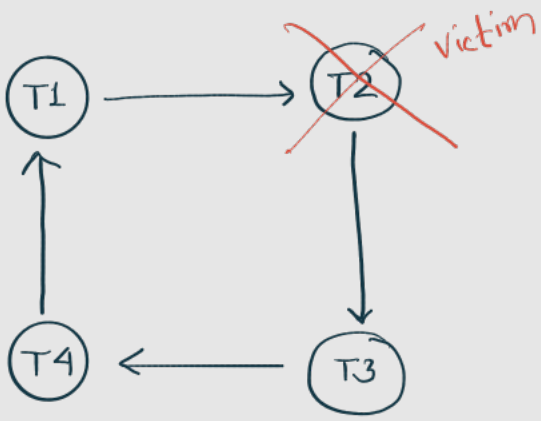


(b) Timeout based deadlock handling:

T1 → A, —
T2 → B, — } Deadlock

Limitation: 1. May even rollback when there is no deadlock.
2. Starvation can occur
3. Choosing correct timeout value is difficult.

③ Recovery:



* Lowest roll back cost → victim

* Problem: Starvation

* Solution:

* Count

* Next time priority $\frac{1}{\text{count}}$

Question:

- ① Suppose transactions T10 (timestamp = 10), T20 (timestamp = 20), and T30 (timestamp = 30) are running concurrently, where a smaller timestamp indicates an older transaction.

T10 (old)

T20

T30 (young)

State what happens (whether the requesting transaction waits or aborts/dies) in both Wait-Die and Wound-Wait schemes for the following requests:

- T10 requests a lock held by T20.
- T30 requests a lock held by T10.

Ans:

a. T10 (Older) requests lock held by T20 (Younger):

- Wait-Die: T10 Waits (older transaction is allowed to wait).
- Wound-Wait: T10 Wounds T20 (T20 is aborted and restarted).

b. T30 (Younger) requests lock held by T10 (Older):

- Wait-Die: T30 Dies (T30 is aborted and restarted).
- Wound-Wait: T30 Waits (younger transaction is allowed to wait for older).

2

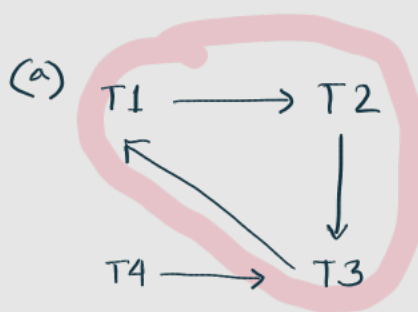
Consider the following lock wait dependencies at a given instance:

- T1 is waiting for a lock held by T2.
- T2 is waiting for a lock held by T3.
- T3 is waiting for a lock held by T1.
- T4 is waiting for a lock held by T3.

a. Represent the Wait-For Graph and determine if a deadlock exists.

b. If a deadlock exists, specify which transactions are involved in the cycle.

c. Explain how the DBMS selects a victim transaction for deadlock recovery and how it prevents starvation.



(b) Deadlock

(c)

a. Graph Edges: T1 -> T2, T2 -> T3, T3 -> T1, and T4 -> T3.

Yes, a deadlock exists because there is a directed cycle: T1 -> T2 -> T3 -> T1.

b. Transactions T1, T2, and T3 are involved in the deadlock cycle. (T4 is waiting on the cycle, but is not part of the cycle itself).

c. The DBMS selects a victim based on the lowest rollback cost, such as the transaction with the fewest updates made or least time executed. To prevent starvation, where the same transaction is repeatedly selected, the DBMS tracks rollback counts, increases the transaction priority after each rollback, and ensures restarted transactions keep their original timestamp.